

California House Price Detector

Task 1-Machine Learning

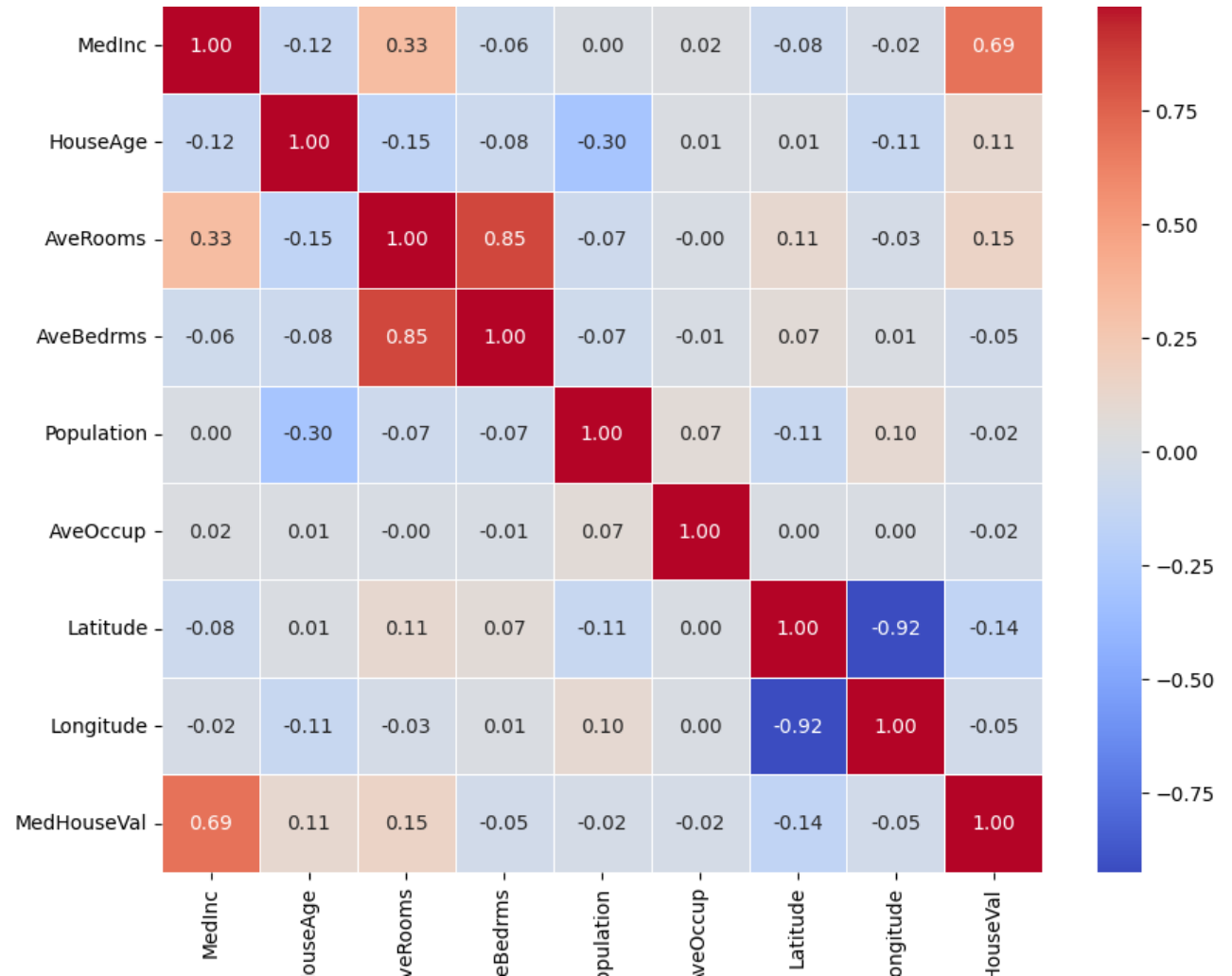
Objective: To build an end-to-end ML workflow-from data loading and EDA to training and evaluation – predicting median house values.

Dataset: California Housing Dataset

- The California Housing Dataset is a popular machine learning dataset available in Scikit-Learn.
- It contains information about housing districts in California collected from the 1990 U.S. Census.
- The goal is to predict the median house value of a district based on various features.
- Key features include:
 - Median Income
 - Housing Median Age
 - Average Rooms
 - Average Bedrooms
 - Population
 - Average Occupancy
 - Latitude
 - Longitude
- This dataset is widely used for regression and predictive modelling tasks.

Exploratory Data Analysis(EDA)

- Loaded the California Housing Dataset using Pandas.
- Examined the dataset structure, columns, and data types.
- Checked for missing values and data inconsistencies.
- Generated summary statistics to understand feature distributions.
- Created visualizations such as histograms and correlation heatmaps.
- Identified relationships between housing features and median house value.



Model Training

- The dataset was divided into training and testing sets using an 80:20 ratio.
- A Linear Regression model was selected for predicting house prices.
- The model was trained using the training dataset.
- Scikit-Learn's Linear Regression library was used for implementation.
- After training, the model was used to predict house prices on the test dataset.
- The performance of the model was evaluated using MAE, RMSE, and R^2 Score

```
import numpy as np
Click to collapse the range. election import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# 1. Feature selection and target splitting
X = df.drop(columns='MedHouseVal')
y = df['MedHouseVal']

# 2. Train/Test Split (This was the missing piece!)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 3. Model Initialization and Training
model = LinearRegression()
model.fit(X_train, y_train)

# 4. Generate Predictions
y_pred = model.predict(X_test)

# 5. Calculate Evaluation Metrics
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

# Print the metrics formatted to 3 decimal places
print(f"Mean Absolute Error (MAE): {mae:.3f}")
print(f"Root Mean Squared Error (RMSE): {rmse:.3f}")
print(f"R2 Score: {r2:.3f}")
```

Evaluation Metrics

- The performance of the Linear Regression model was evaluated using regression metrics.
- Mean Absolute Error (MAE): Measures the average difference between actual and predicted house prices.
- Root Mean Squared Error (RMSE): Measures prediction error while giving more importance to larger errors.
- R^2 Score: Indicates how well the model explains the variation in house prices.
- Lower MAE and RMSE values and a higher R^2 score indicate better model performance.

Model Performance:

- MAE=0.533
- RMSE=0.746
- R^2 Score=0.576

Mean Absolute Error (MAE): 0.533

Root Mean Squared Error (RMSE): 0.746

R^2 Score: 0.576

Results

- The Linear Regression model was successfully trained and evaluated.
- The model was able to predict house prices using housing features.
- The evaluation metrics showed reasonable predictive performance.
- The project successfully demonstrated the complete machine learning workflow.
- Linear Regression provides a strong baseline for house price prediction.

Future Improvements

- Perform feature engineering to improve model accuracy.
- Handle outliers and missing values more effectively.
- Tune model parameters for better performance.
- Experiment with advanced algorithms such as Random Forest Regression and XGBoost.
- Develop a user-friendly interface for real-time house price prediction.

THANK YOU